

Social Media NLP Analysis Report

Mia Emanuele

Table of contents

Describing and defining an application of data science using unstructured data.....	2
NLTK Implementation and Output	4
Figure 1.....	10
Justification and explanation of NLTK	10
Textblob Implementation and Output	11
Justification and explanation of TextBlob.....	13
Comparison of NLTK and TextBlob.....	14
References.....	15

Describing and defining an application of data science using unstructured data

I prompted ChatGPT to generate an example of unstructured social media text that could indicate a potential public order issue within Coventry city centre, including the Coventry University campus, together with two shorter related examples. The generated examples were fictional and created solely for the purpose of demonstrating Natural Language Processing (NLP) techniques

Prompt: ‘Provide me with an example of unstructured social media text that could indicate a potential public order issue in Coventry city centre which includes Coventry university campus. Then 2 relative but shorter examples’.

Output:

Below are **fictional example posts** of unstructured social media text that could suggest a potential public order issue in Coventry city centre. These are illustrative only.

Longer Example (Unstructured Post)

“Big link up tonight 🔥
8:30 @ Broadgate by the statue
bring fireworks 😂 don’t be soft
cov uni lot better show up
#CovTakeover #LinkUp #CityCentre”

Two Shorter Examples

1. “Meet 8pm by the statue. Don’t be late”
2. “#CovTakeover”

If you’d like, I can also:

- Provide examples tailored for training data (e.g., more ambiguous vs. more explicit signals),
- Create labelled examples (e.g., *Low / Medium / High risk indicator*), or
- Generate examples reflecting different tones (sarcastic, coded language, youth slang, etc.).

Unstructured data, such as text, images and audio, does not follow a predefined format or schema (Krantz et al., 2025). Unlike structured data, which can be easily organised into rows and columns within databases or spreadsheets, unstructured data is far more complex to process due to its variability and lack of consistent organisation. Common examples include social media posts, audio recordings and image files, all of which contain valuable information but require specialist analytical techniques to extract meaningful insights. Through data science methods such as Natural Language Processing (NLP) and machine learning, unstructured data can be transformed into structured information suitable for analysis and decision-making (Krantz et al., 2025).

Natural Language Processing follows a structured pipeline consisting of sentence segmentation, tokenisation, stopword removal, stemming, lemmatisation, dependency parsing and part-of-speech (POS) tagging. Each stage contributes to converting raw text into structured linguistic information that can be analysed computationally. Application areas include customer insight, behavioural analysis, artificial intelligence and public safety intelligence.

One practical application of NLP is the analysis of social media posts to identify potential indicators of antisocial behaviour, public disorder or threats to public safety. Social media platforms are frequently used to coordinate gatherings, communicate locations and mobilise groups through hashtags, abbreviations and informal language. Without automated text analysis techniques, these early indicators may be overlooked. Consequently, NLP can support the early identification of patterns that may warrant further human investigation.

The fictional social media post analysed within this report provides an example of this application:

“Big link up tonight 🔥
8:30 @ Broadgate by the statue
bring fireworks 🤪 don't be soft
cov uni lot better show up
#CovTakeover #LinkUp #CityCentre”

This example contains several characteristics commonly associated with unstructured social media text, including slang ("don't be soft"), informal abbreviations ("cov" and "uni"), hashtags and emojis. Although the post does not explicitly state an intention to commit a criminal offence, its meaning becomes more informative when considered alongside similar posts, such as "Meet 8pm by the statue. Don't be late" or messages containing hashtags such as "#CovTakeover". Individually these posts may appear ambiguous; however, when analysed collectively they may indicate the potential organisation of a large public gathering involving fireworks and youth mobilisation.

Following analysis using the NLP pipeline, the extracted linguistic information could assist neighbourhood policing teams in making informed operational decisions. For example, officers may increase patrols, monitor CCTV coverage around identified locations or position response units appropriately. Importantly, the NLP analysis should be viewed as a decision-support tool that provides contextual information for human assessment rather than as a system that independently determines criminal intent.

NLTK Implementation and Output

```
import nltk
```

```
nltk.download('stopwords')
nltk.download('words')
nltk.download('vader_lexicon')
```

```
post_text = ''' Big link up tonight 🔥
                8:30 @ Broadgate by the statue
                bring fireworks 😂 don't be soft
                cov uni lot better show up
                #CovTakeover #LinkUp #CityCentre '''
```

```
# Following the NLP pipeline:
```

```
# -----
```

```
# Sentence segmentation
```

```
# -----
```

```
from nltk.tokenize import sent_tokenize
post_sent = sent_tokenize(post_text)
print(post_sent)
```

```
print('\n')
```

```
# -----
```

```
# Tokenisation
```

```
# -----
```

```
from nltk.tokenize import word_tokenize
post_words = word_tokenize(post_text) # turns the text into list of words
print(post_words)
```

```
print('\n')
```

```
# -----
```

```
# Filtering stop words
```

```

# -----
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))

post_filtered = []
for word in post_words:
    if word.lower() not in stop_words:
        post_filtered.append(word) # checking if given word is in this set
print(post_filtered)

print('\n')

# -----

# Stemming

# -----
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()

for word in post_filtered:
    root = stemmer.stem(word) # looking closer at stemmed words
    if (root!=word):
        print(word, ' --> ', root)

print('\n')

# -----

# Tagging parts of speech (POS)

# -----
post_pos = nltk.pos_tag(word_tokenize(post_text))
print(post_pos)
# ^ labelling words in sentence according to their POS eg 'noun'

print('\n')

# -----

# Lemmatising

```

```

# -----
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

def translate_pos(tag):
    """Convert NN/VB/JJ/RB to n/v/a/r (translating in
    order to apply lemmatiser to POS tags)."""
    if tag.startswith('J'):
        result = 'a'
    elif tag.startswith('V'):
        result = 'v'
    elif tag.startswith('R'):
        result = 'r'
    else:
        result = 'n'
    return(result)

for sentence in post_sent:
    post_pos = nltk.pos_tag(word_tokenize(sentence))
    for word,pos in post_pos:
        print(lemmatizer.lemmatize(word,translate_pos(pos)),end=' ')

print('\n')

# -----

# Named entity recognition

# -----
post_pos = nltk.pos_tag(word_tokenize(post_text))
tree = nltk.ne_chunk(post_pos)
print(tree)
# ^identifying phrases (small sequence of tokens) with some meaning

grammar = "NP: {<DT>?<JJ>*<NN>}"

chunk_parser = nltk.RegexpParser(grammar)
tree = chunk_parser.parse(post_pos)
tree.draw()

print('\n')

```

```
# -----

# Sentiment analysis

# -----
from nltk.sentiment import SentimentIntensityAnalyzer
sia = SentimentIntensityAnalyzer()

score = sia.polarity_scores(post_text)
print(score)
```

Output:

```
[' Big link up tonight 🔥 \n      8:30 @ Broadgate by the statue \n      bring
fireworks 😂 don't be soft \n      cov uni lot better show up \n      #CovTakeover
#LinkUp #CityCentre']
```

```
['Big', 'link', 'up', 'tonight', '🔥', '8:30', '@', 'Broadgate', 'by', 'the', 'statue', 'bring',
'fireworks', '😂', 'don', "'", 't', 'be', 'soft', 'cov', 'uni', 'lot', 'better', 'show', 'up', '#',
'CovTakeover', '#', 'LinkUp', '#', 'CityCentre']
```

```
['Big', 'link', 'tonight', '🔥', '8:30', '@', 'Broadgate', 'statue', 'bring', 'fireworks', '😂', "'",
'soft', 'cov', 'uni', 'lot', 'better', 'show', '#', 'CovTakeover', '#', 'LinkUp', '#', 'CityCentre']
```

Big --> big

Broadgate --> broadgat

statue --> statu

fireworks --> firework

CovTakeover --> covtakeov

LinkUp --> linkup

CityCentre --> citycentr

[('Big', 'JJ'), ('link', 'NN'), ('up', 'RP'), ('tonight', 'NN'), ('🔥', 'RB'), ('8:30', 'CD'), ('@', 'JJ'), ('Broadgate', 'NNP'), ('by', 'IN'), ('the', 'DT'), ('statue', 'NN'), ('bring', 'NN'), ('fireworks', 'NNS'), ('😂', 'VBP'), ('don', 'JJ'), ('"', 'NNP'), ('t', 'NN'), ('be', 'VB'), ('soft', 'JJ'), ('cov', 'NNS'), ('uni', 'JJ'), ('lot', 'NN'), ('better', 'RBR'), ('show', 'VBP'), ('up', 'RP'), ('#', '#'), ('CovTakeover', 'NNP'), ('#', '#'), ('LinkUp', 'NNP'), ('#', '#'), ('CityCentre', 'NNP')]

Big link up tonight 🔥 8:30 @ Broadgate by the statue bring firework 😂 don ' t be
soft cov uni lot well show up # CovTakeover # LinkUp # CityCentre

(S

Big/JJ

link/NN

up/RP

tonight/NN

🔥/RB

8:30/CD

@/JJ

Broadgate/NNP

by/IN

the/DT

statue/NN

bring/NN

fireworks/NNS

😂/VBP

don/JJ

'/NNP

t/NN

be/VB

soft/JJ

cov/NNS

uni/JJ

lot/NN

better/RBR

show/VBP

up/RP

##

CovTakeover/NNP

##

(ORGANIZATION LinkUp/NNP)

##

(ORGANIZATION CityCentre/NNP))

{'neg': 0.0, 'neu': 0.884, 'pos': 0.116, 'compound': 0.4404}

Parse tree generated using NLTK of the analysed social media post



Following tokenisation, stopwords removal eliminates common words that contribute little semantic value, while stemming reduces lexical variation by converting related words to their root forms. For example, "fireworks" is reduced to "firework" and "CityCentre" becomes "citycentr". This standardisation improves the ability to detect recurring terms and behavioural patterns across multiple social media posts, supporting scalable analysis rather than relying solely on manual interpretation.

Figure 4 presents the syntactic parse tree generated by NLTK, illustrating the grammatical relationships between words through noun phrases, verb phrases and prepositional phrases.

In addition, named-entity recognition (NER) identifies "LinkUp" and "CityCentre" as organisations. Although these hashtags do not represent formal organisations, their identification demonstrates NLTK's ability to recognise recurring named entities within the text. Even where classifications are imperfect, entity recognition can assist in identifying repeated hashtags, locations or event names across multiple posts, potentially highlighting coordinated activity.

Finally, the sentiment analysis produced scores of neg = 0.0, neu = 0.884, pos = 0.116 and compound = 0.4404, indicating a predominantly neutral message with a moderately positive tone. The absence of strongly negative sentiment suggests that the post does not contain explicit aggression or direct threats, while the positive score reflects enthusiasm or anticipation. Within a policing context, this suggests that the message is more consistent with the organisation of a social gathering than with overtly violent or extremist intent.

Overall, the NLTK analysis provides structured linguistic evidence relating to event organisation, time and location references, and directive language. Rather than relying solely on subjective interpretation, the NLP pipeline extracts objective textual features that can support consistent and proportionate public order risk assessment.

Textblob Implementation and Output

```
from textblob import TextBlob

# -----

# Creating TextBlob

# -----
post = TextBlob(""" Big link up tonight 🔥
8:30 @ Broadgate by the statue
bring fireworks 😂 don't be soft
cov uni lot better show up
#CovTakeover #LinkUp #CityCentre """)

# -----

# Part-of-Speech (POS) tagging

# -----
print(post.tags) # labelling words in sentence according to their POS eg 'noun'

print('\n')

# -----

# Sentiment analysis

# -----
print(post.sentiment) # checking polarity and subjectivity
print(post.sentiment.polarity)

print('\n')
```

```

# -----

# Tokenisation

# -----
print(post.words) # breaking TextBlob into words

print('\n')

print(post.sentences) # breaking TextBlob into sentences

print('\n')

for sentence in post.sentences:
    print(sentence.sentiment)

print('\n')

# -----

# Parsing

# -----
print(post.parse())

```

Output:

```

[('Big', 'JJ'), ('link', 'NN'), ('up', 'RP'), ('tonight', 'NN'), ('🔥', 'RB'), ('8:30', 'CD'), ('@', 'JJ'),
('Broadgate', 'NNP'), ('by', 'IN'), ('the', 'DT'), ('statue', 'NN'), ('bring', 'NN'), ('fireworks',
'NNS'), ('😂', 'VBP'), ('don', 'JJ'), ('', 'NNP'), ('t', 'NN'), ('be', 'VB'), ('soft', 'JJ'), ('cov', 'NNS'),
('uni', 'JJ'), ('lot', 'NN'), ('better', 'RBR'), ('show', 'VBP'), ('up', 'RP'), ('CovTakeover', 'NNP'),
('LinkUp', 'NNP'), ('CityCentre', 'NNP')]

```

```

Sentiment(polarity=0.19999999999999998, subjectivity=0.31666666666666665)

```

```

0.19999999999999998

```

['Big', 'link', 'up', 'tonight', '🔥', '8:30', 'Broadgate', 'by', 'the', 'statue', 'bring', 'fireworks', '😂', 'don', "'", 't', 'be', 'soft', 'cov', 'uni', 'lot', 'better', 'show', 'up', 'CovTakeover', 'LinkUp', 'CityCentre']

[Sentence(" Big link up tonight 🔥

8:30 @ Broadgate by the statue

bring fireworks 😂 don't be soft

cov uni lot better show up

#CovTakeover #LinkUp #CityCentre")]

Sentiment(polarity=**0.19999999999999998**, subjectivity=**0.31666666666666665**)

Big/NNP/B-NP/O link/NN/I-NP/O up/IN/B-PP/O tonight/RB/B-ADVP/O 🔥/NN/B-NP/O **8:30**/CD/O/O @/IN/B-PP/B-PNP Broadgate/NNP/B-NP/I-PNP by/IN/B-PP/B-PNP the/DT/B-NP/I-PNP statue/NN/I-NP/I-PNP bring/VB/B-VP/O fireworks/NNS/B-NP/O 😂/NN/I-NP/O don/VB/B-VP/O '/NN/B-NP/O t/NN/I-NP/O be/VB/B-VP/O soft/JJ/B-NP/O cov/NN/I-NP/O uni/NN/I-NP/O lot/NN/I-NP/O better/JJR/B-NP/O show/NN/I-NP/O up/IN/B-PP/O #/##/O/O CovTakeover/NN/B-NP/O #/##/O/O LinkUp/NN/B-NP/O #/##/O/O CityCentre/NN/B-NP/O

Justification and explanation of TextBlob

TextBlob analyses text by evaluating grammatical structure, sentiment and linguistic patterns. Similar to NLTK, its part-of-speech (POS) tagging assigns grammatical labels to individual words, identifying proper nouns representing locations, action-oriented verbs such as "*bring*" and "*show*", and descriptive adjectives including "*soft*". Collectively, these grammatical features reinforce the interpretation that the post contains directive language intended to encourage participation in a planned gathering.

Furthermore, sentiment analysis produced a polarity score of approximately 0.20 and a subjectivity score of 0.32, indicating a slightly positive and moderately objective tone. The positive polarity suggests enthusiasm or anticipation rather than overt hostility, while the moderate subjectivity reflects a combination of factual information, such as the specified time and location, alongside expressive language. Consequently, the sentiment analysis supports the interpretation that the post promotes attendance at a social gathering rather than communicating explicit threats or aggressive intent.

In addition, sentence detection and syntactic parsing demonstrate that the message follows a coherent grammatical structure despite containing informal language, slang, emojis and hashtags. Rather than representing fragmented or random text, the analysis identifies meaningful phrases such as "*bring fireworks*" and "*better show up*", both of which convey instruction and encouragement. These linguistic patterns strengthen the interpretation that the post is intended to coordinate attendance and promote participation in an organised event.

Overall, TextBlob combines grammatical analysis with sentiment evaluation to provide a structured interpretation of unstructured social media text. By identifying linguistic features alongside emotional tone, the library supports a more objective assessment of online communication and provides contextual information that may assist human decision-making during the preliminary assessment of potential public order risks.

Comparison of NLTK and TextBlob

While both NLTK and TextBlob successfully analyse the fictional social media post, they primarily rely on lexicon-based and rule-based approaches. Although these methods provide a valuable foundation for text preprocessing and initial linguistic analysis, their effectiveness is limited when applied to real-world social media content. Informal language, slang, emojis, sarcasm and evolving online terminology are often difficult to interpret accurately using predefined linguistic rules alone. Consequently, I would recommend using these libraries as tools for structured preprocessing and preliminary risk assessment rather than as standalone solutions for operational decision-making.

For more robust real-world applications, I would recommend complementing these libraries with more advanced NLP frameworks. Honnibal and Montani (2017) describe spaCy as an industrial-strength NLP library specifically designed for practical applications. Compared with NLTK, spaCy offers improved processing efficiency and more accurate handling of informal language, named-entity recognition and contextual relationships. These capabilities make it better suited to analysing the complexity of social media communication.

Furthermore, scikit-learn provides supervised machine learning algorithms capable of classifying text based on previously labelled examples (Pedregosa et al., 2011). When integrated with spaCy, machine learning models can distinguish between low-risk social gatherings and higher-risk events by learning patterns directly from data rather than relying solely on predefined linguistic rules. This enables more adaptive, scalable and context-aware classification.

Overall, combining spaCy with scikit-learn would provide a more robust solution than relying exclusively on NLTK or TextBlob. Such an approach would improve entity recognition, contextual understanding and classification accuracy while remaining scalable for larger volumes of social media data. Within a policing context, these tools should be viewed as decision-support technologies that assist analysts by highlighting potentially relevant posts, rather than replacing professional judgement or determining intent independently.

References

- Honnibal, & Montani. (2017, January). *spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing*. Sentometrics Research.
- Krantz, T., Jonker, A., & Gomstyn, A. (2025, November 10). *Unstructured data examples applications and use cases*. Ibm.com. <https://www.ibm.com/think/topics/unstructured-data-examples-use-cases>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Louppe, G., Prettenhofer, P., Weiss, R., Weiss, R. J., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.*, 12. <https://doi.org/10.5555/1953048.2078195>